

TRENTOS-M RPi_SPI_LED

- [Resources](#)
 - [Driver Code](#)
 - [Datasheets](#)
- [Overview](#)
- [LED Interface implementation](#)
- [Limitation](#)
- [Hardware](#)
 - [Raspberry Pi SPI interfaces](#)
 - [MAX7219 LED SPI interface](#)
- [Usage](#)
 - [CMake](#)
 - [CMakeES](#)
- [Platforms](#)

Resources

Driver Code

- Driver for BCM2837 SPI peripheral access: <https://www.airspayce.com/mikem/bcm2835/index.html>
- Driver for MAX7219 access: <https://os.mbed.com/components/MAX7219-64-LED-Display-Driver-or-8-Digit/>

Datasheets

- MAX7219 chip: <https://datasheets.maximintegrated.com/en/ds/MAX7219-MAX7221.pdf>
- 1088AS 8x8 LED display: https://www.velleman.eu/downloads/29/infosheets/vmp502_led8x8_1088as.pdf

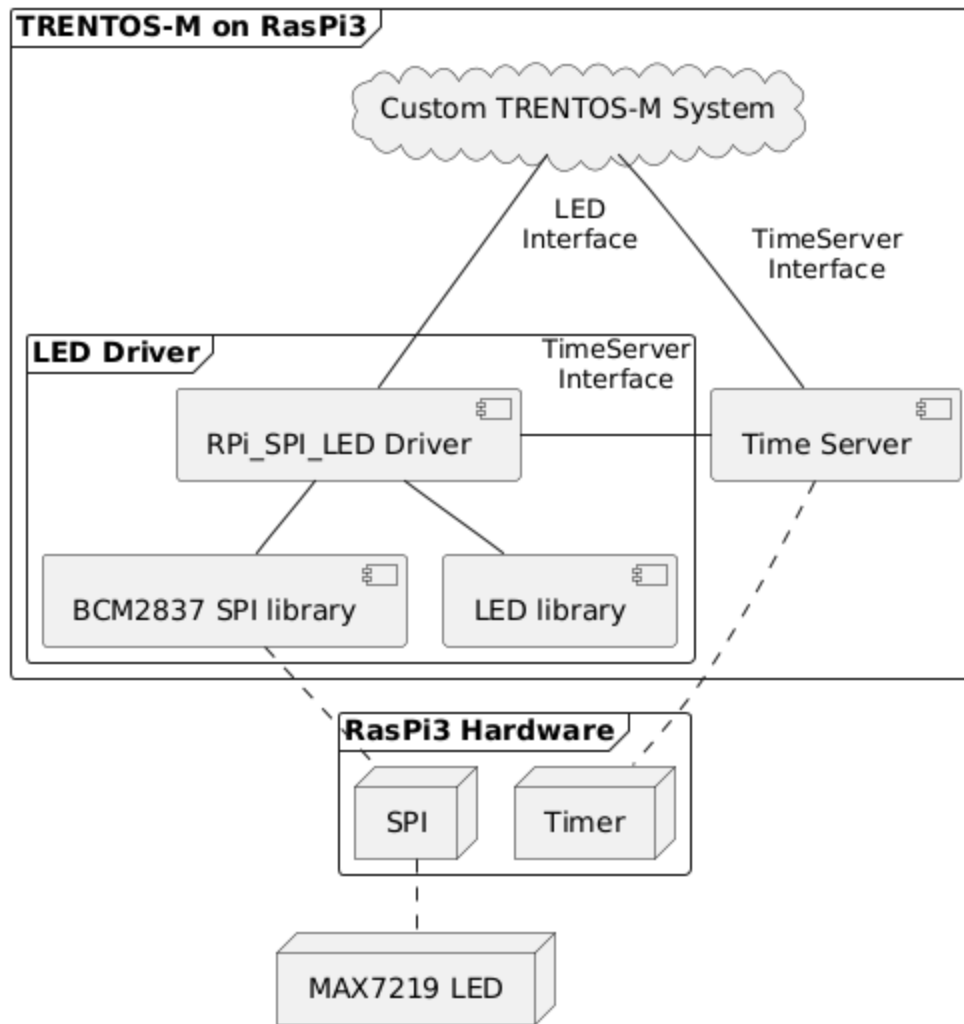
Overview

The RPi_SPI_LED component is basically a combination of a redesigned version of the RPi_SPI_Flash component and a 3rd party LED C++ library. The MAX7219 chip is connected to the Raspberry Pi over SPI and enables communication with an 8x8 LED dot-matrix display (1088AS).

A 3rd party library is used in the driver for BCM2837 SPI peripheral access and MAX7219 access.

The driver (*RPI_SPI_LED.c*) provides the TRENTOS-M LED interface on top, which can be used directly by applications. Furthermore, the driver links SPI calls from the MAX7219 LED library to the BCM2837 library by implementing some callback functions (*impl_spiled_spi_writenb()*, *impl_spiled_spi_cs()*, *impl_spiled_wait()*). SPI functions in the MAX7219 LED library are linked to these functions in the *post_init()* function.

The driver and the MAX7219 LED library need to implement waits, thus timer functionality needs to be provided which makes it necessary to implement a TimeServer component.



RPi_SPI_LED Driver - Architecture

LED Interface implementation

The driver's implementation of the LED interface provides currently four different functions that can be used by a specific application:

- *clear_display()*: clears the entire LED display.
- *display_char_on_device()*: displays a given character on a specific device.
- *display_char()*: displays a given character on all connected MAX7219 devices.
- *scroll_text()*: scrolls a given string across the LED display.

Internally the display functions use a lookup table to get the corresponding bit pattern for a given character. The scroll function shifts the corresponding bit pattern of the different characters in the message bitwise through a buffer.

Limitation

Currently the display driver implements only very basic functionality and is specifically tailored towards the 8x8 LED display. Furthermore, the scrolling operation can be implemented in a more reusable way. In the long term, the interface should be redesigned. The idea would be that a frame buffer and a viewport are provided. According functions must be implemented like *drawPixel()*, *drawFont()*, *moveViewport()* and *dumpViewport()*. The according image/animation would be drawn into the framebuffer (*drawPixel()*, *drawFont()*) and the viewport would be moved accordingly (*moveViewport()*). After moving the viewport, the according data in the framebuffer can be drawn onto the actual LED display (*dumpViewport()*). That way an application would gain more control over animations. This framebuffer/viewport concept could be extended to multiple virtual framebuffers that are combined into one actual framebuffer for more complex animations.

Hardware

Raspberry Pi SPI interfaces

General information regarding SPI usage on the RPi can be found in the official RPi SPI documentation: <https://www.raspberrypi.org/documentation/hardware/raspberrypi/spi/README.md>

In order to be able to use SPI, a potential device has to be connected and wired to the respective Pins accordingly. See also <https://www.raspberrypi.org/documentation/usage/gpio> or <https://de.pinout.xyz/pinout/spi> for the assignment. Note that the numbering on the 2x20 GPIO pin headers is different to the BCM2738 SoC's internal I/O numbering. The documentation often refers to the SoC's internal I/O signals when describing the I/O functions, while for actually connecting wires and peripherals physically, referring to the 2x20 GPIO headers is more practical.

Signal		RasPI GPIO 2x20 Pin Header	BCM2837 SoC internal I/O
SPI0	MOSI	19	10
	MISO	21	9
	SCLK	23	11
	CE0	24	8
	CE1	26	7
SPI1	MOSI	38	20
	MISO	35	19
	SCLK	40	21
	CE0	12	18
	CE1	11	17
	CE2	36	16

MAX7219 LED SPI interface

In order to use a SPI-based LED display, the respective device has to be wired to the RPi first. Exemplarily, the respective mapping for a MAX7219 chip connected at SPI0 is shown in the table below:

MAX7219 - Pin Name	Raspberry Pi - Pin Name	Raspberry Pi - Physical Pin
VCC	5V	PIN 2, 4
GND	GND	PIN 6, 9, 14, 20, 25, 30, 34, 39
DIN	GPIO 10 (MOSI)	PIN 19
CS	GPIO 8 (SPI CE0)	PIN 24
CLK	GPIO 11 (SPI CLK)	PIN 23

Better compare these values again with help of the recommended connection (<https://codingworld.io/project/8x8-led-matrix-anschliessen-und-programmieren>) and Raspberry Pi pinout (<https://www.raspberrypi.org/documentation/usage/gpio/>).

The following image shows the LED display connected to the Raspberry Pi:

TODO: take picture and upload it

Usage

CMake

RPI_SPI_LED Driver - Usage 1
<pre> DeclareCMakeSCComponent_RPI_SPI_LED(<TypeNameLED> LIBS <TimeServerClient>)</pre>

CAMkES

RPI_SPI_LED Driver - Usage 2

```
#include "RPI_SPI_LED/camkes/RPI_SPI_LED.camkes"
DECLARE_COMPONENT_RPI_SPI_LED_DRV(<TypeNameLED>)

#include "TimeServer/camkes/TimeServer.camkes"
DECLARE_COMPONENT_TimeServer(<TypeTimeServer>)

assembly{
  composition{
    component <TypeMyApplication> <InstanceMyApplication>;

    DECLARE_AND_CONNECT_INSTANCE_RPI_SPI_LED(
      <TypeNameLED>, <InstanceNameLED>
    )

    connection sel4RPCall conn(from <InstanceMyApplication.rpc>, to <InstanceNameLED.rpc>);

    DECLARE_AND_CONNECT_INSTANCE_TimeServer(
      <TypeTimeServer>, <InstanceTimeServer>,
      <InstanceNameFlash>.timeServer_rpc
    )
  }

  configuration{
    CONFIGURE_INSTANCE_RPI_SPI_LED(<InstanceNameLED>)
  }
}
```

Platforms

- RPi3
- RPi4 (branch: rpi4_porting)